

That's a really good question. Once you understand this, the whole `json.Decoder` API will make much more sense.

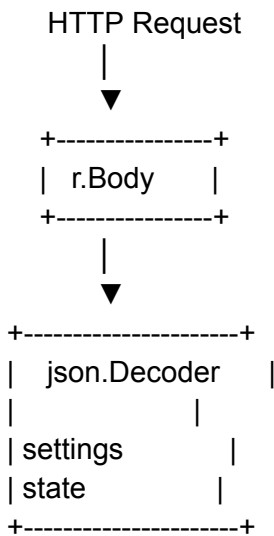
Step 1: `json.NewDecoder(r.Body)`

When you write:

```
decoder := json.NewDecoder(r.Body)
```

Go creates a **Decoder object**.

Think of it like creating a machine.



Notice something.

No JSON has been read yet.

You're just building the decoder.

Step 2:

decoder.DisallowUnknownFields()

Now you do

```
decoder.DisallowUnknownFields()
```

This **doesn't decode anything**.

It simply changes one setting inside the decoder.

Imagine the decoder internally looks like this:

```
type Decoder struct {  
    reader io.Reader  
  
    disallowUnknownFields bool  
}
```

Initially

```
decoder.disallowUnknownFields = false
```

After calling

```
decoder.DisallowUnknownFields()
```

it becomes

```
decoder.disallowUnknownFields = true
```

Nothing else happens.

No JSON has been read.

Step 3: `decoder.Decode(&student)`

Now the decoder starts reading.

Suppose the JSON is

```
{  
  "name":"Suyash",  
  "email":"abc@gmail.com",  
  "age":23,  
  "college":"IIT"  
}
```

The decoder reads it token by token.

```
{  
"name"  
"Suyash"  
"email"  
"abc@gmail.com"  
"age"  
23  
"college"  
"IIT"  
}
```

Step 4: It matches every field

It looks inside your struct

```
type Student struct {  
    Name string `json:"name"`  
    Email string `json:"email"`  
    Age int `json:"age"`  
}
```

Internally it has something like

Known fields

"name"

"email"

"age"

Now it starts matching.

Reads

"name"

Found.

Store

student.Name = "Suyash"

Reads

"email"

Found.

Store

student.Email = "abc@gmail.com"

Reads

"age"

Found.

Store

student.Age = 23

Then it reads

"college"

Now it asks

Do I know this field?

Known fields:

name

email

age

No.

Case 1: Default behavior

If

`decoder.disallowUnknownFields == false`

Go simply says

I don't know this field.

Skip it.

So it ignores

`"college": "IIT"`

and continues.

Case 2: After `DisallowUnknownFields()`

Now

```
decoder.DisallowUnknownFields == true
```

When it reaches

```
"college"
```

it says

```
Unknown field.
```

```
My settings say this is forbidden.
```

Immediately it returns

```
json: unknown field "college"
```

and stops decoding.

Why doesn't `DisallowUnknownFields()` return a decoder?

You might think it should work like this:

```
json.NewDecoder(r.Body).  
    DisallowUnknownFields().  
    Decode(&student)
```

But `DisallowUnknownFields()` is a **configuration method**.

It modifies the existing decoder.

Think of it like configuring a washing machine.

```
washer := NewWasher()
```

```
washer.SetTemperature(60)  
washer.SetSpinSpeed(1000)
```

washer.Start()

`SetTemperature()` doesn't create another washer.

It changes the settings of the current washer.

Exactly the same thing happens here.

decoder := json.NewDecoder(r.Body)

decoder.DisallowUnknownFields()

decoder.Decode(&student)

Real-life analogy

Imagine an airport security officer.

You first create the officer.

Officer

Then you tell him

Reject fake passports.

He remembers that rule.

Later passengers arrive.

Passenger 1

✓ Valid passport

Allowed.

Passenger 2

✗ Fake passport

Officer remembers the rule and rejects them.

`DisallowUnknownFields()` is like giving the officer instructions **before** the passengers arrive.

This pattern—**create an object** → **configure it** → **use it**—is very common in Go. You'll see it with `json.Decoder`, HTTP clients, database connections, scanners, and many other types in the standard library.